Feed-forward neural networks for regression and classification: the Runge function and the MNIST handwritten digits

Alberto García Asensio*
Department of Physics, University of Oslo, Norway
(Dated: November 9, 2026)

Neural networks have become one of the most widely used tools in machine learning, but their flexibility comes at a price: many parameters, many hyperparameters, and a non-convex cost function. In this work, we investigate when that price is worth paying by comparing a dense feed-forward neural network (FFNN), which we implemented ourselves using backpropagation, with simpler methods on two problems. For the regression of the noisy Runge function, the network achieves a cross-validation MSE of 0.012, close to the noise level of 0.01. This is comparable to degree-12 OLS and Ridge, but much more stable at the edges of the interval, where the polynomial blows up when extrapolating. We compare standard gradient descent, RMSprop and Adam; sigmoid, ReLU and Leaky ReLU activations; different depths and widths; and L_1 and L_2 penalties, selecting all hyperparameters using cross-validation. Adam is the most robust optimizer, the learning rate is the most important hyperparameter, and the penalties have little effect. For MNIST digit classification, a network with one hidden layer of 200 ReLU neurons achieves a test accuracy of 97.7 %, compared with 92.7 % for logistic regression, and agrees with scikit-learn (97.8 %). We conclude that, for a smooth one-dimensional function, a well-chosen polynomial is inexpensive and difficult to outperform, whereas a neural network is worth its cost when the data features are not known in advance.

I. INTRODUCTION

Artificial neural networks are now the basis of many advances in machine learning, from image and speech recognition to applications in physics, such as experimental data analysis and the solution of differential equations [3]. Their theoretical appeal comes from the universal approximation theorem: a network with a single hidden layer and enough neurons can approximate any continuous function to any desired accuracy [9]. However, the theorem states that approximation is possible, not that it is easy to achieve. A network has many more parameters than a linear model, its cost function is non-convex, and its performance depends on many choices: the architecture, activation function, optimizer, learning rate, and regularization [3, 4]. It is therefore important to understand when a network is worth using instead of simpler and better-understood methods.

In Project 1 [2], we fitted the noisy Runge function [6] using polynomials, with OLS, Ridge [7], and Lasso [8]. In that case, we chose the model features ourselves: the powers $x, x^2, \ldots, x^p$. A neural network does not require this choice, because it constructs its own features in the hidden layers from the input x. In this work, we compare both approaches on the same regression problem, and then move on to a problem in which the appropriate features are not known in advance: the classification of handwritten digits from the MNIST database [19, 20].

To do this, we implemented our own dense neural network, with a flexible number of layers and neurons and different activation functions, and calculated its gradient using the backpropagation algorithm [10]. We trained it using standard gradient descent and stochastic gradient descent with RMSprop and Adam, and selected the hyperparameters using cross-validation or a validation set. We compared the results with OLS, Ridge, and Lasso for regression, and with logistic regression and the scikit-learn implementation [21] for classification.

The report is organized as follows. In Sec. II, we present the data, cost and activation functions, the neural network and backpropagation, the optimizers, the model-selection method, and the metrics. In Sec. II H, we describe our implementation and the code tests. In Sec. III, we present and discuss the results: first, the regression of the Runge function (Secs. III A–III D), and then MNIST classification (Sec. III E). In Sec. IV, we provide a critical evaluation of all the methods, and in Sec. V, we present our conclusions.

II. METHODS

A. Data sets

For regression, we use exactly the same data as in Project 1 [2]. We generate $n = 100$ equally spaced points x_i in $[-1, 1]$ and add Gaussian noise to them,

$$y_i = f(x_i) + \varepsilon_i, \qquad f(x) = \frac{1}{1 + 25x^2}, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{1}$$

with $\sigma = 0.1$ and random seed 2026. Since the noise cannot be predicted, the lowest MSE that any model can achieve on new data is $\sigma^2 = 0.01$. We split the data into 80 % for training (80 points) and 20 % for testing (20 points), using the same seed as in Project 1. Unlike in Project 1, the network input is not a design matrix containing powers of x, but simply x itself, as a single column. There is no need to construct the polynomial,

because the network forms its own features in the hidden layers. We scale the input by subtracting the mean and dividing by the standard deviation of the training set, so that the inputs to the sigmoid functions are of order one.

For classification, we use the MNIST database [20]: 70 000 grayscale images of handwritten digits, each measuring 28×28 pixels and belonging to one of 10 classes (0 to 9), with the standard split into 60 000 training images and 10 000 test images. Each image is converted into a 784-component vector, and its values are scaled to the interval $[0, 1]$ by dividing by 255. From the 60 000 training images, we randomly set aside 50 000 for training and 10 000 as a validation set, which we use to select the hyperparameters. To save computational time, the hyperparameter searches are performed using 20 000 of the training images, as allowed by the assignment. The test set is used only once, at the end, to evaluate the selected models. It therefore does not influence any of our decisions, and its result provides an unbiased estimate of the error on new data.

B. Cost functions and their gradients

We write the network output for sample i as $\boldsymbol{a}_i^L$, the activation of the last layer L, and its linear pre-activation as $\boldsymbol{z}_i^L = \boldsymbol{a}_i^{L-1} W^L + \boldsymbol{b}^L$, with samples stored in the rows of the input as in the week-41 template. In Project 1 the model $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$ was linear in the parameters, and the gradient of the cost with respect to $\boldsymbol{\theta}$ could be written in closed form, $\nabla_{\boldsymbol{\theta}} \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2 = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})$. The output of a neural network is a nested, non-linear function of all the weights, so this is no longer possible. What a cost function has to provide is only its derivative with respect to the output of the network; the chain rule, organised as the back propagation algorithm (Section II D), carries it back to every weight and bias. The cost functions of neural networks are discussed in Section 8.7 of the lecture notes [1].

Mean squared error (regression). For a single output and n samples we use

$$C_{\mathrm{MSE}} = \frac{1}{2n} \sum_{i=1}^{n} \left(a_i^L - y_i\right)^2 . \tag{2}$$

The factor $\frac{1}{2}$ does not move the minimum; it only removes the factor 2 of the derivative, which is equivalent to rescaling the learning rate γ by a factor two in plain gradient descent (and is irrelevant for RMSprop and Adam, which divide the gradient by a running estimate of its magnitude). The factor $1/n$ makes the cost an average, so that the gradient with respect to the weights, which sums the contributions of all samples, is a mean and does not grow with the number of samples or the size of the minibatch. Only the term $i = j$ depends on a_j^L, so

$$\frac{\partial C_{\mathrm{MSE}}}{\partial a_j^L} = \frac{1}{n}\left(a_j^L - y_j\right). \tag{3}$$

For regression the output layer is the identity, $a^L = z^L$, and this is also the error of the output layer, $\delta_j^L = \partial C / \partial z_j^L = (a_j^L - y_j)/n$.

Binary cross entropy (binary classification). For labels $y_i \in \{0, 1\}$ the output layer has one neuron with the sigmoid activation, and $a_i^L = \sigma(z_i^L)$ is read as the probability $P(y_i = 1 \mid \boldsymbol{x}_i)$. The probability of the observed label is the Bernoulli distribution $P(y_i \mid \boldsymbol{x}_i) = (a_i^L)^{y_i}(1 - a_i^L)^{1-y_i}$, and minimising the negative logarithm of the likelihood of n independent samples gives the cost (Section 5.3 of the lecture notes)

$$C_{\mathrm{BCE}} = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \ln a_i^L + (1 - y_i) \ln\left(1 - a_i^L\right) \Big]. \tag{4}$$

For $y_i = 1$ only $-\ln a_i^L$ survives, which vanishes for $a_i^L = 1$ and diverges for $a_i^L \to 0$; for $y_i = 0$ the roles are exchanged. Confident wrong predictions are penalised much more than uncertain ones. Differentiating,

$$\frac{\partial C_{\mathrm{BCE}}}{\partial a_j^L} = -\frac{1}{n}\left[\frac{y_j}{a_j^L} - \frac{1 - y_j}{1 - a_j^L}\right] = \frac{1}{n}\,\frac{a_j^L - y_j}{a_j^L\left(1 - a_j^L\right)}. \tag{5}$$

With $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$ (Section II C) the denominator cancels against the derivative of the output activation,

$$\delta_j^L = \frac{\partial C_{\mathrm{BCE}}}{\partial a_j^L}\,\sigma'(z_j^L) = \frac{1}{n}\left(a_j^L - y_j\right). \tag{6}$$

This is the reason why the mean squared error is not used with a sigmoid output for classification: it would give $\delta^L = \frac{1}{n}(a - y)\,a(1 - a)$, and for a confident wrong prediction such as $y = 1$, $a = 0.001$, the factor $a(1 - a) \approx 10^{-3}$ makes the error signal about a thousand times smaller than with the cross entropy, so that the network learns slowest exactly where it is most wrong.

Multiclass cross entropy with the softmax (Sections 5.6 and 8.7). For K classes the output layer has K neurons, whose pre-activations $z_{i1}, \ldots, z_{iK}$ (the logits) are turned into probabilities by the softmax function,

$$a_{ik} = \frac{e^{z_{ik}}}{\sum_{m=1}^{K} e^{z_{im}}}, \qquad k = 1, \ldots, K. \tag{7}$$

Every a_{ik} is positive and $\sum_k a_{ik} = 1$. The label is one-hot encoded, $y_{ik} = 1$ for the correct class and 0 otherwise, and the cost is

$$C_{\mathrm{CE}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \ln a_{ik}, \tag{8}$$

where for each sample the inner sum leaves only minus the logarithm of the probability assigned to the correct class; for $K = 2$ it reduces to Eq. (4). To differentiate, we call c the correct class of sample j and insert Eq. (7), which gives $C_j = -\frac{1}{n}\ln\left(e^{z_{jc}}/\sum_m e^{z_{jm}}\right) = \frac{1}{n}\left(-z_{jc} + \ln\sum_m e^{z_{jm}}\right)$. The derivative of $-z_{jc}$ with respect to z_{jm} is -1 for $m = c$ and 0 otherwise, that is $-y_{jm}$, and the derivative of the logarithm is $e^{z_{jm}}/\sum_{m'} e^{z_{jm'}} = a_{jm}$. Hence

$$\delta^L_{jm} = \frac{\partial C_{\mathrm{CE}}}{\partial z^L_{jm}} = \frac{1}{n}\left(a_{jm} - y_{jm}\right), \tag{9}$$

and the Jacobian of the softmax never has to be implemented. Since the components of both $\boldsymbol{a}_j$ and $\boldsymbol{y}_j$ sum to one, the components of $\boldsymbol{\delta}^L_j$ sum to zero: what one class gains in probability, the others lose. Two practical remarks. First, the softmax is unchanged if the same constant c is subtracted from all logits, $e^{z_k-c}/\sum_m e^{z_m-c} = e^{-c}e^{z_k}/(e^{-c}\sum_m e^{z_m}) = a_k$; choosing $c = \max_m z_m$ prevents overflow of the exponentials. Second, if all logits are equal the network assigns $1/K$ to every class and the cost is $\ln K$, which for MNIST ($K = 10$) gives $\ln 10 \approx 2.30$: the value an untrained network should start from.

Regularisation. Following Section 8.12 of the lecture notes, an L_2 or an L_1 penalty on all the weights can be added to any of the three costs,

$$C \to C + \lambda\sum_l \left\|W^l\right\|_2^2 \quad \text{or} \quad C \to C + \lambda\sum_l \left\|W^l\right\|_1, \tag{10}$$

where the norms are the sums of the squares or of the absolute values of all the elements of each weight matrix. Their derivatives are

$$\frac{\partial}{\partial W^l_{pq}}\lambda\sum_l \left\|W^l\right\|_2^2 = 2\lambda W^l_{pq}, \tag{11}$$

$$\frac{\partial}{\partial W^l_{pq}}\lambda\sum_l \left\|W^l\right\|_1 = \lambda\,\mathrm{sign}\left(W^l_{pq}\right), \tag{12}$$

where for the L_1 norm we use the subgradient 0 at $W^l_{pq} = 0$, where $|w|$ is not differentiable. The penalties do not depend on the output, so they leave δ^L unchanged and only add $2\lambda W^l$ or $\lambda\,\mathrm{sign}(W^l)$ to the gradient of each weight matrix. The biases are not penalised: a bias only shifts the activation of a neuron and does not control how strongly the fitted function can vary, which is what the penalty is meant to limit.

C. Activation functions

In every hidden layer the linear combination $\boldsymbol{z}^l = \boldsymbol{a}^{l-1}W^l + \boldsymbol{b}^l$ is passed through a function f, applied element by element, $\boldsymbol{a}^l = f(\boldsymbol{z}^l)$ (Section 8.6 of the lecture notes and the lecture material of week 41). The activation has to be non-linear: without it two layers collapse into one, $(\boldsymbol{x}W^1+\boldsymbol{b}^1)W^2+\boldsymbol{b}^2 = \boldsymbol{x}(W^1W^2)+(\boldsymbol{b}^1W^2+\boldsymbol{b}^2)$, and a network of any depth is a single linear map. Its derivative enters the back propagation of the error from layer $l+1$ to layer l, $\boldsymbol{\delta}^l = (\boldsymbol{\delta}^{l+1}W^{l+1,T}) \odot f'(\boldsymbol{z}^l)$ (Section II D), so that the error is multiplied by f' every time it crosses a hidden layer. We test three activations.

Sigmoid (Section 8.6.1).

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big). \tag{13}$$

The derivative follows from $\sigma'(z) = e^{-z}/(1+e^{-z})^2$ and $1-\sigma(z) = e^{-z}/(1+e^{-z})$. The sigmoid maps the real line to $(0,1)$; its derivative is largest at $z = 0$, where $\sigma'(0) = \frac{1}{4}$, and it vanishes for large $|z|$, where the function saturates. Since $\sigma' \le \frac{1}{4}$, an error propagated back through L sigmoid layers is reduced by at least a factor 4^L (to about 10^{-3} of its size after five layers): the gradients of the first layers vanish and these layers hardly learn (Section 8.9).

ReLU (Section 8.6.2).

$$\mathrm{ReLU}(z) = \max(0,z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases} \tag{14}$$

At $z = 0$ the function is not differentiable and we use the value 0. In the active region the derivative is exactly one, so the error passes back without being reduced, and the function is cheap to evaluate. A neuron whose pre-activation is negative for all inputs has zero gradient and can never recover (a "dead" neuron).

Leaky ReLU (Section 8.6.2).

$$\mathrm{LReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \le 0, \end{cases} \tag{15}$$

$$\mathrm{LReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z < 0, \end{cases} \tag{16}$$

with a small fixed slope, here $\alpha = 0.01$. The derivative is never zero, so neurons with negative pre-activations still receive a small gradient, which avoids dead neurons while keeping the unit derivative of the ReLU in the active region.

D. Feed-forward neural network and back propagation

A dense network consists of layers of neurons, and each neuron in a layer receives the outputs of all neurons in the previous layer. We follow the notation in the week 41 course material: samples are stored in the rows of the input matrix $\boldsymbol{X}$, the weight matrix W^l of layer l has shape (inputs, outputs), and $\boldsymbol{b}^l$ is the bias vector. A network with layer sizes $[n_0, n_1, \ldots, n_L]$ has L layers with parameters, and layer l has $n_{l-1}n_l$ weights and n_l biases. In the

TABLE I. The three cost functions with their output activations. In all three cases the error of the output layer has the same form, because each cost is paired with its natural output activation.

Problem	Output activation	Cost	$\boldsymbol{\delta}^L = \partial C/\partial \boldsymbol{z}^L$
Regression	identity	Eq. (2)	$(\boldsymbol{a}^L - \boldsymbol{y})/n$
Binary classification	sigmoid	Eq. (4)	$(\boldsymbol{a}^L - \boldsymbol{y})/n$
Multiclass classification	softmax	Eq. (8)	$(\boldsymbol{a}^L - \boldsymbol{y})/n$

TABLE II. The three activation functions of the hidden layers.

Activation	$f(z)$	$f'(z)$	Weakness
Sigmoid	$1/(1+e^{-z})$	$\sigma(1-\sigma) \le \frac{1}{4}$	vanishing gradients
ReLU	$\max(0, z)$	1 $(z>0)$, 0 $(z<0)$	dead neurons
Leaky ReLU	z $(z>0)$, αz $(z\le 0)$	1 $(z>0)$, α $(z<0)$	extra parameter α

forward pass, each layer calculates a linear combination of the activations from the previous layer and applies the activation function to it. In the hidden layers, we use the sigmoid, ReLU, or Leaky ReLU functions, and in the output layer we use the identity function for regression and softmax for classification.

For training, we need the gradient of the cost with respect to all weights and biases, which we calculate using the four backpropagation equations (Secs. 8.8.3 to 8.8.5 of Ref. [1], and Refs. [5, 10]). Eq. (18) is the output-layer error, which, according to Sec. II B, has the same form for all three cost functions. Eq. (19) propagates the error backwards, from layer $l+1$ to layer l, through the same weights that connect them, multiplied by the derivative of the activation function. Eqs. (20) and (21) give the gradients of the weights and biases, summing the contributions from all samples. These equations are the chain rule organized so that one forward pass and one backward pass provide the gradient with respect to all parameters, at a cost of approximately two forward passes, regardless of the number of parameters.

Feed-forward pass, for $l = 1, \ldots, L$ with $\boldsymbol{a}^0 = \boldsymbol{X}$:

$$\boldsymbol{z}^l = \boldsymbol{a}^{l-1} W^l + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = f(\boldsymbol{z}^l). \tag{17}$$

Back propagation:

$$\boldsymbol{\delta}^L = \frac{1}{n}\left(\boldsymbol{a}^L - \boldsymbol{y}\right), \tag{18}$$

$$\boldsymbol{\delta}^l = \left(\boldsymbol{\delta}^{l+1}(W^{l+1})^T\right) \odot f'(\boldsymbol{z}^l), \tag{19}$$

$$\frac{\partial C}{\partial W^l} = (\boldsymbol{a}^{l-1})^T \boldsymbol{\delta}^l, \tag{20}$$

$$\frac{\partial C}{\partial \boldsymbol{b}^l} = \sum_{i=1}^{n} \boldsymbol{\delta}^l_{i\cdot}. \tag{21}$$

The initial weights are drawn from a normal distribution with zero mean and standard deviation $\sigma_W = \sqrt{1/n_{\text{in}}}$, where n_{in} is the number of inputs to the layer (Sec. 8.10 of Ref. [1], and Ref. [11]). The reason is that, for a neuron with n_{in} independent inputs, $\text{Var}(z) = n_{\text{in}}\, \sigma_W^2\, \mathbb{E}[a^2]$, which grows with the number of inputs. With $\sigma_W^2 = 1/n_{\text{in}}$, the variance of z is of order one, regardless of the layer size. If it were much larger, the sigmoid functions would be saturated and the gradient would vanish from the beginning; if it were much smaller, the signals and gradients would be very small. The weights must be random to break the symmetry between neurons: if they all started with the same values, all neurons in a layer would calculate the same thing and receive the same gradient, and the network would behave as if it had only one neuron per layer. The biases do not need to be random, and we initialize them to the small value 0.01. For ReLU, He initialization is recommended, with $\sigma_W^2 = 2/n_{\text{in}}$ [12], because ReLU deactivates approximately half of the neurons. Since our networks are shallow, we use the same initialization for all three activation functions.

E. Optimisers

To minimize the cost, we use the gradient-descent methods from Project 1 [2]. In standard gradient descent, each step uses all the training data, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \gamma \nabla C$, where γ is the learning rate and $\boldsymbol{\theta}$ represents any weight matrix W^l or bias vector $\boldsymbol{b}^l$. In stochastic gradient descent (SGD), the data are shuffled at each epoch and divided into mini-batches of M samples, and a step is taken using the gradient of each mini-batch. An epoch is one complete pass through the data, so with n samples there are n/M steps per epoch. RMSprop [17] divides the step by a moving average of the squared gradients,

$$v_t = \rho v_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\gamma\, g_t}{\sqrt{v_t} + \epsilon}, \tag{22}$$

and Adam [18] also combines a moving average of the gradients (the momentum) with a correction for the initial bias,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{23}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{24}$$

$$\theta_{t+1} = \theta_t - \gamma\, \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \epsilon}, \tag{25}$$

where all operations are element-wise. The difference from Project 1 is that these rules are now applied separately to each W^l and $\boldsymbol{b}^l$, and, above all, that the cost is no longer convex: it can have several minima and flat regions, so the result depends on the initialization, learning rate, and optimizer.

We use the standard values $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ from Ref. [18], and $\rho = 0.9$ for RMSprop. For regression, we use mini-batches of $M = 10$ samples, so with 80 training points there are 8 steps per epoch. We train for 500 epochs with RMSprop and Adam, which is enough to reach the noise level (Sec. III A). Standard gradient descent takes only one step per epoch, so we use 3000 epochs to make the number of steps comparable. For MNIST, we use mini-batches of 64 images and, due to computational time, 10 epochs for the hyperparameter searches and 20 for the final models.

F. Model selection

We select all hyperparameters using only the training data, so that the test set remains independent. For regression, we use cross-validation with $K = 5$ folds on the 80 training points: the data are shuffled and split into five folds of 16 points, each model is trained five times using four folds and evaluated on the remaining fold, and the input scaling is fitted using only the training folds to prevent data leakage. The cross-validation error and its standard error are

$$\mathrm{CV}_K = \frac{1}{K}\sum_{k=1}^{K}\mathrm{MSE}_k, \tag{26}$$

$$\mathrm{SE}_K = \frac{1}{\sqrt{K}}\sqrt{\frac{\sum_{k=1}^{K}\left(\mathrm{MSE}_k - \mathrm{CV}_K\right)^2}{K-1}}. \tag{27}$$

We use the same folds for the network and the polynomials, so the comparison is fair. We select the value of each hyperparameter that gives the lowest CV MSE. For the (γ, λ) grids, we apply the one-standard-error rule: among the values whose CV MSE is within one standard error of the minimum, we choose the one with the largest λ, i.e., the simplest model, because the data do not allow us to distinguish between these models (Sec. 3.14 of Ref. [1], and Ref. [4]). The selected model is then retrained using all 80 training points and evaluated once on the test set. For MNIST, training each combination five times would be too computationally expensive, so we use the 10 000-image validation set described in Sec. II A.

G. Metrics

For regression, we measure the quality of the fit using the mean squared error, $\mathrm{MSE} = \frac{1}{n}\sum_i (y_i - \tilde{y}_i)^2$. The lower it is, the better; however, on new data it can never fall below the noise level $\sigma^2 = 0.01$: a test MSE close to 0.01 means that the model has captured the function and only the noise remains. If the training MSE falls clearly below σ^2, the model is fitting the noise, which indicates overfitting.

For classification, we use accuracy, the fraction of correctly classified images,

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} I(t_i = y_i), \tag{28}$$

where I is the indicator function, which is equal to 1 if the prediction y_i matches the label t_i and 0 otherwise. It takes values between 0 and 1. Random guessing would give 0.1 for ten classes, logistic regression on the pixels achieves approximately 0.92, and convolutional neural networks exceed 0.99 [19]; these are the reference values against which we compare our results. Since accuracy is a proportion measured over n images, its standard error is $\sqrt{p(1-p)/n}$.

The confusion matrix C has, in element C_{ij}, the number of images from class i classified as j. The diagonal contains the correct predictions, while the off-diagonal elements show which digits are confused with one another, something that accuracy alone cannot reveal. From it, we calculate the precision, recall, and their harmonic mean F_1 for each class c (Sec. 5.12 of Ref. [1]),

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c},$$
$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}, \tag{29}$$

where TP, FP, and FN are the true positives, false positives, and false negatives. All three metrics are equal to 1 in the ideal case. Low precision for a class means that many images from other classes are assigned to it, while low recall means that many images belonging to that class are not recognized.

H. Implementation and tests

All the code is written in Python. The neural network, forward pass, backpropagation, cost and activation functions, optimizers (standard gradient descent, RMSprop, and Adam), cross-validation for the network, and logistic regression (our network without hidden layers) are our own implementations using NumPy [22]. We use scikit-learn [21] for the training/test split, scaling, `KFold` folds, Ridge and Lasso for comparison with Project 1, classification metrics (accuracy, confusion matrix, precision, recall, and F_1), and `MLPClassifier` as a reference network. The figures are made with Matplotlib [23]. All results use random seed 2026 and can be reproduced using the code in our GitHub repository,https://github.com/albertgq1/FYS-STK3155-Project2.

We performed three tests to check the code (Table III). First, we compared the backpropagation gradient with centered finite differences in a small $[1, 5, 3, 1]$ network: the largest relative error is 4.2×10^{-7}, which is of the expected order for finite differences and confirms that the four equations are implemented correctly. Second, the initial cost of an untrained network on MNIST is 2.336, very close to $\ln 10 = 2.303$, the value corresponding to assigning equal probability to all ten classes. Third, our network and scikit-learn's `MLPClassifier`, with the same architecture, optimizer, and hyperparameters, achieve test accuracies of 0.9770 and 0.9784, respectively, which agree within the standard error of 0.0015.

TABLE III. **Tests of the code.** Result of each test; the expected value is given in parentheses where it applies.

Test	Result
Gradient vs. finite diff.	4.2×10^{-7}
Initial cost, MNIST	2.336 ($\ln 10 = 2.303$)
Accuracy, ours / scikit-learn	0.9770 / 0.9784

III. RESULTS AND DISCUSSION

A. Learning rate and optimisers

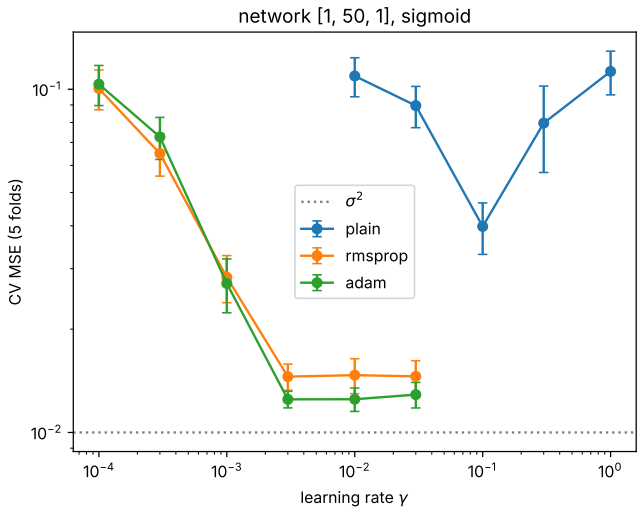


FIG. 1. **Choice of the learning rate.** Cross-validated MSE (5 folds) as a function of the learning rate γ for the network $[1, 50, 1]$ with sigmoid activation, trained with plain gradient descent (3000 epochs, full batch), RMSprop and Adam (500 epochs, minibatches of 10). The error bars are the standard errors and the dotted line is the noise level $\sigma^2 = 0.01$.

According to theory, we expect standard gradient descent, which takes a single step per epoch with a fixed learning rate, to work only within a narrow range of γ: if γ is small, it progresses very slowly, and if it is large, it oscillates or diverges. RMSprop and Adam normalize the step by the size of recent gradients, so they should be much less sensitive to γ, as we already observed in Project 1 [2].

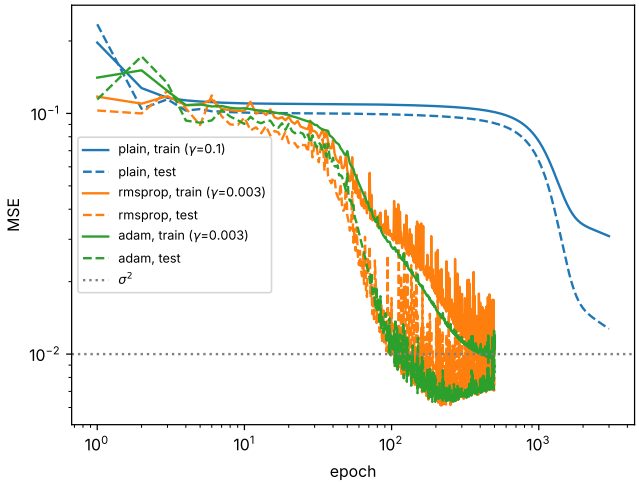


FIG. 2. **Training curves of the three optimisers.** Training MSE (solid) and test MSE (dashed) as a function of the epoch for plain gradient descent, RMSprop and Adam with their best learning rate, network $[1, 50, 1]$. The dotted line is σ^2.

Fig. 1 confirms this picture. Standard gradient descent only gives acceptable results near $\gamma = 0.1$, and even there the CV MSE (0.040) is four times the noise level, because 3000 steps are not enough to converge; with $\gamma = 0.3$ and $\gamma = 1$, the error increases again because the steps are too large. RMSprop and Adam achieve a CV MSE between 0.0125 and 0.0147 throughout the γ interval from 3×10^{-3} to 3×10^{-2}, close to $\sigma^2 = 0.01$, whereas for $\gamma \leq 10^{-3}$ they are too slow for 500 epochs. Adam performs slightly better than RMSprop, with a minimum of 0.0125 ± 0.0007.

The training curves in Fig. 2 show why. Standard gradient descent decreases the cost very slowly. RMSprop is very noisy because it has no momentum term, whereas Adam, which averages the gradients, is smoother. Both adaptive methods reach the noise level in about 100 to 200 epochs. Therefore, we use Adam with $\gamma = 3 \times 10^{-3}$ for the remainder of the regression analysis.

B. Network architecture compared with OLS

Table IV compares the four networks required by the assignment with the best model from Project 1, degree-12 OLS. The four networks have a CV MSE between 0.0117 and 0.0158, and their results are within one or two standard errors of one another, even though the largest network has about 70 times more parameters than the

TABLE IV. **Networks compared with OLS.** Networks with one and two hidden layers of 50 and 100 nodes (sigmoid, Adam, $\gamma = 3\times10^{-3}$, 500 epochs) and OLS with degree 12, the best model of project 1: number of parameters, cross-validated MSE on the same 5 folds, training MSE and test MSE.

Model	Param.	CV MSE	Train	Test
$[1, 50, 1]$	151	0.0125 ± 0.0007	0.0119	0.0092
$[1, 100, 1]$	301	0.0158 ± 0.0028	0.0114	0.0089
$[1, 50, 50, 1]$	2701	0.0120 ± 0.0016	0.0112	0.0101
$[1, 100, 100, 1]$	10401	0.0117 ± 0.0010	0.0100	0.0080
OLS, degree 12	12	0.311 ± 0.294	0.0110	0.0061

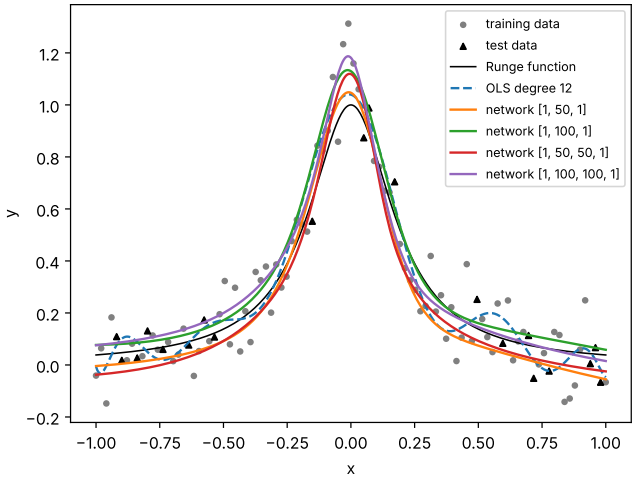


FIG. 3. **Fitted functions.** The four networks and OLS with degree 12, all trained on the 80 training points, together with the Runge function, the training data and the test data.

smallest. For this smooth one-dimensional function, a larger network does not clearly improve the fit.

OLS has the lowest test MSE (0.0061), but the test set contains only 20 points, and the uncertainty in its MSE, of order $\sigma^2\sqrt{2/20} \approx 0.003$, is comparable to the differences between models. By contrast, its CV MSE is 0.31 ± 0.29, because one of the five folds gives an MSE of 1.49. This is the only fold in which the endpoint $x = 1$ is in the validation set, so the degree-12 polynomial has to extrapolate beyond its training data and blows up, which is precisely the Runge phenomenon [6]; in the other four folds, its MSE is between 0.014 and 0.023. Fig. 3 illustrates this: all models follow the peak of the function well, but the polynomial oscillates near the edges, whereas the networks remain smooth. We conclude that the network and OLS are comparable for this problem: OLS is much less expensive, while the network is more stable near the edges because it does not behave like a high-degree polynomial.

C. Activation functions, depth and overfitting

According to theory (Sec. II C), the sigmoid saturates for large $|z|$ and its derivative is at most 1/4. ReLU has derivative 1 in the active region, but its neurons can die if their input is always negative [13], and Leaky ReLU avoids this problem by having a small slope α in the negative region [14].

First, we select the learning rate for each activation function using cross-validation on the $[1, 50, 1]$ network: $\gamma = 3\times10^{-3}$ for the sigmoid and $\gamma = 10^{-2}$ for ReLU and Leaky ReLU. With $\gamma = 3 \times 10^{-2}$, both ReLU variants become unstable (CV MSE ≈ 0.025), because, unlike the sigmoid, they are not bounded. Fig. 4 shows the CV MSE and training MSE for networks with 1 to 4 hidden layers and 10 to 100 neurons per layer. The best CV MSE is 0.0116 for the sigmoid, with 4 layers of 25 neurons, and 0.0127 and 0.0126 for ReLU and Leaky ReLU, respectively, with 2 layers of 25 neurons. Most values in each heatmap lie between 0.012 and 0.016, so depth and width have little effect on the result. The sigmoid performs slightly better on this problem because it is a smooth function, like the Runge function, whereas ReLU approximates the curve using straight-line segments; Leaky ReLU behaves almost identically to ReLU.

The bottom row of Fig. 4 shows signs of overfitting: as the number of layers increases, the training MSE falls to 0.007 or 0.008 with ReLU, clearly below $\sigma^2 = 0.01$, while the CV MSE does not improve. A training MSE below the noise level means that the network is fitting the noise in the data. This is clearer in Fig. 5, where we train the $[1, 100, 100, 1]$ network for 5000 epochs. The training MSE falls to about 0.006, while the validation MSE reaches its minimum very early (at epoch 67 with ReLU and Leaky ReLU, and at epoch 411 with the sigmoid) and then rises to 0.024–0.037. If training is stopped at this minimum, a technique known as early stopping [16], the number of epochs acts as a form of regularization. This is the same bias-variance story as in Project 1: there, complexity was controlled by the polynomial degree, whereas here it is controlled by the number of epochs.

Fig. 6 illustrates the vanishing-gradient problem. In a network with 6 hidden layers of 50 neurons, at the beginning of training, the gradient in the first layer is about 10^{-5} times the gradient in the last hidden layer with the sigmoid, but only about 10^{-1} times with ReLU and Leaky ReLU. The reason is Eq. (19): each time the error is propagated backwards through a layer, it is multiplied by $f'(z)$, which is at most 1/4 for the sigmoid, so after L layers the gradient is reduced by at least a factor of 4^L [15] (Sec. 8.9 of Ref. [1]). With ReLU, the derivative is 1 for active neurons, and the gradient reaches the first layers. In deep networks, this means that the first layers barely learn with the sigmoid, and it is the main reason for using the ReLU family. In our networks with 1 to 4

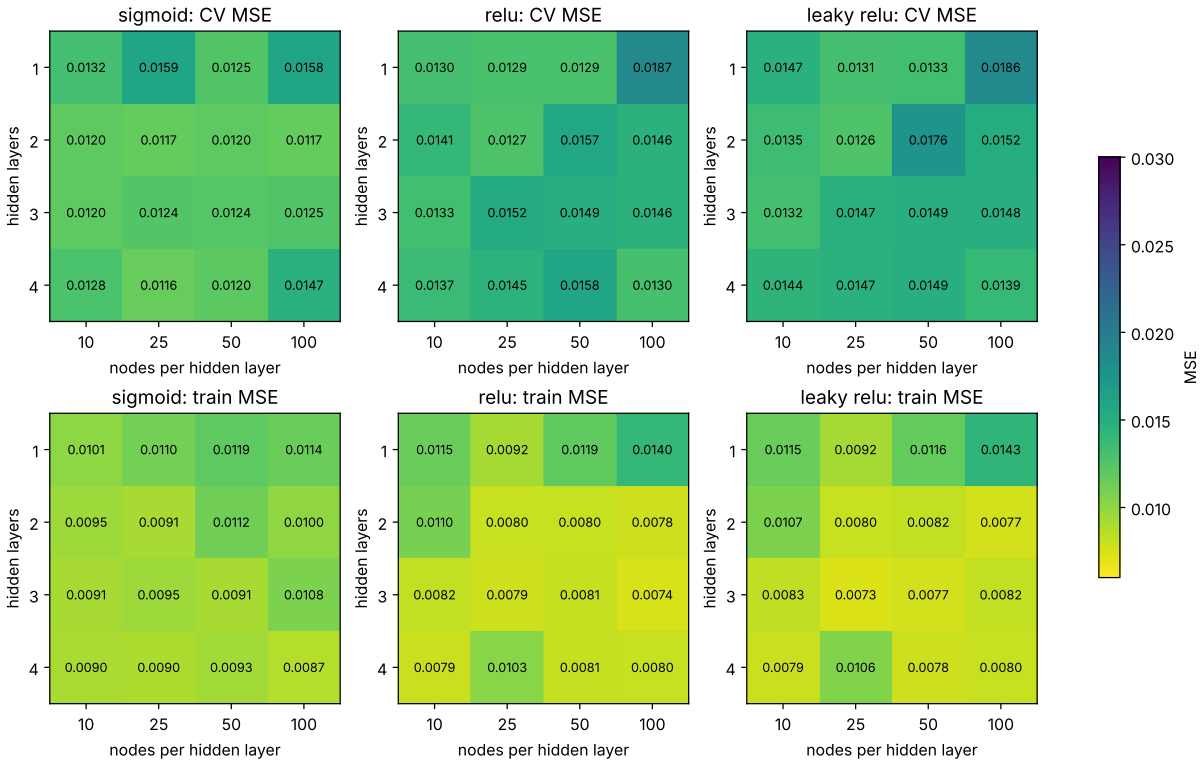


FIG. 4. **Effect of the activation function, the depth and the width.** Top: cross-validated MSE; bottom: training MSE, as a function of the number of hidden layers (1–4) and of nodes per layer (10–100) for the sigmoid, ReLU and Leaky ReLU (Adam with the best γ of each activation, 500 epochs).

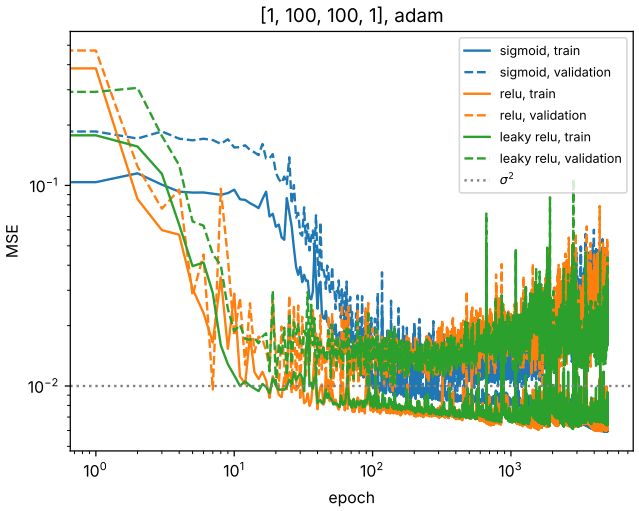


FIG. 5. **Overfitting with many epochs.** Training and validation MSE as a function of the epoch for the network $[1, 100, 100, 1]$ with the three activation functions (5000 epochs, 64 training and 16 validation points). The dotted line is σ^2.

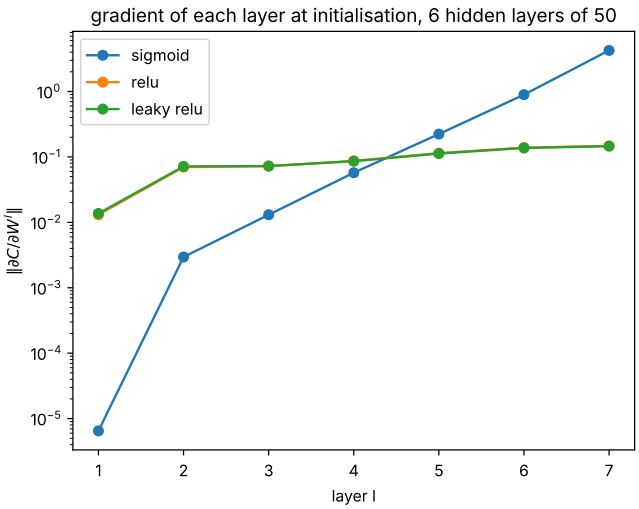


FIG. 6. **Vanishing gradients.** Norm of the gradient of the weights of each layer at initialisation, for a network with 6 hidden layers of 50 nodes and the three activation functions.

layers, the effect is smaller, which is why the sigmoid still works well.

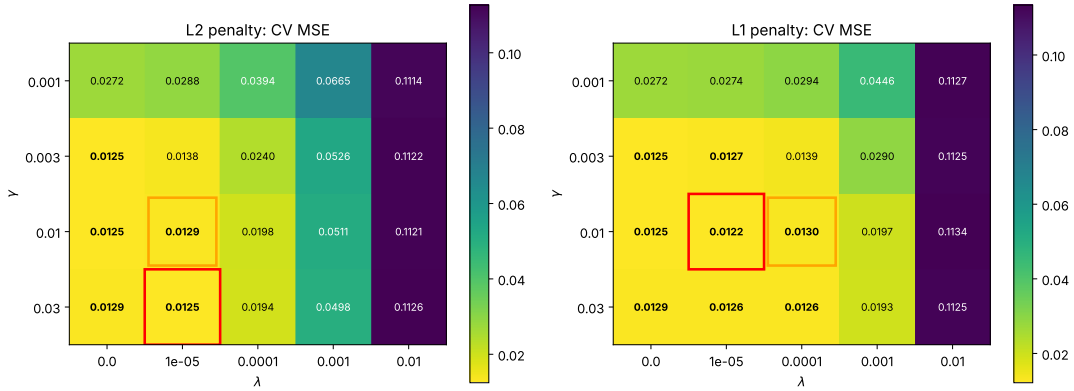


FIG. 7. **Learning rate and penalty.** Cross-validated MSE of the network $[1, 50, 1]$ (sigmoid, Adam, 500 epochs) as a function of γ and λ for the L_2 (left) and L_1 (right) penalties. Bold: cells within one standard error of the minimum (red box); orange box: the one-standard-error choice.

TABLE V. **Penalised networks compared with Ridge and Lasso.** Networks with L_2 and L_1 penalty and Ridge and Lasso with degree 12 (project 1), with the same data and the same folds. The test set is only used here.

Model	λ	CV MSE	Test
NN $[1, 25, 25, 1]$, L_2	10^{-5}	0.0124 ± 0.0016	0.0076
Ridge, degree 12	10^{-5}	0.0296 ± 0.0084	0.0083
NN $[1, 25, 1]$, L_1	10^{-4}	0.0124 ± 0.0010	0.0083
Lasso, degree 12	10^{-3}	0.0455 ± 0.0080	0.0093

D. L2 and L1 penalties compared with Ridge and Lasso

According to theory (Sec. II B), the L_2 penalty shrinks all weights in proportion to their size, whereas L_1 pushes them towards zero with a constant force, just as in Ridge and Lasso. Since we observed little overfitting after 500 epochs in Sec. III C, we expect the optimal value of λ to be small.

Fig. 7 confirms this. For $\lambda \leq 10^{-5}$ (L_2) or $\lambda \leq 10^{-4}$ (L_1), the CV MSE is the same as without a penalty, within the error bars. For larger λ, the fit becomes worse because the weights are pushed towards zero and the network becomes too simple; with $\lambda = 10^{-2}$, the CV MSE is about 0.11, the variance of the data, meaning that the network only predicts the mean. L_2 already worsens the fit at $\lambda = 10^{-4}$, whereas L_1 does not, because the two penalties have different scales. Applying the one-standard-error rule, we choose $\gamma = 0.01$ with $\lambda = 10^{-5}$ for L_2 and $\lambda = 10^{-4}$ for L_1: these are the most regularized models that the data do not allow us to distinguish from the minimum, and we therefore prefer them because they are simpler. Using these values, we compare several architectures; the best are $[1, 25, 25, 1]$ with L_2 and $[1, 25, 1]$ with L_1, both with a CV MSE of 0.0124.

Table V compares these networks with degree-12 Ridge and Lasso, evaluated using the same data and the same folds. In cross-validation, the penalized networks (0.0124) are clearly better than Ridge (0.0296) and Lasso (0.0455). This is mainly because the polynomials once again have large errors in folds where an endpoint of the interval is in the validation set, whereas the networks have very similar errors in all folds. On the test set, all models are between 0.0076 and 0.0093, close to σ^2, and with only 20 points these differences are of the order of the noise.

The values of λ cannot be compared directly between the network and the polynomials. Our network uses the cost as an average over the samples, whereas scikit-learn's Ridge and Lasso use it as a sum; furthermore, the folds differ from those in Project 1, where we used 10 folds over the 100 points. Lasso performs worse than Ridge, probably because scikit-learn's coordinate-descent algorithm converges poorly with the powers $x, x^2, \ldots, x^{12}$, which are highly correlated. As in Project 1 [2], where the best λ for Ridge and Lasso was also very small, regularization has little effect on this problem, because with 80 points and a smooth function there is little overfitting to correct.

E. Classification of MNIST

TABLE VI. **Final MNIST models.** Test accuracy (test set used once) of the models trained on the 60 000 training images for 20 epochs, with the standard error $\sqrt{p(1-p)/n}$.

Model	Test accuracy
Logistic regression (ours)	0.9265 ± 0.0026
NN $[784, 200, 10]$, ReLU (ours)	0.9770 ± 0.0015
`MLPClassifier` [200]	0.9784 ± 0.0015

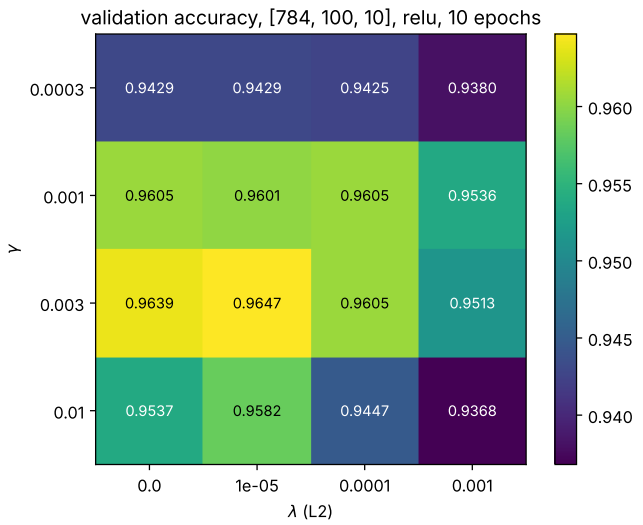


FIG. 8. **Learning rate and penalty for MNIST.** Validation accuracy of the network $[784, 100, 10]$ with ReLU as a function of γ and λ (L_2), trained on 20 000 images for 10 epochs.

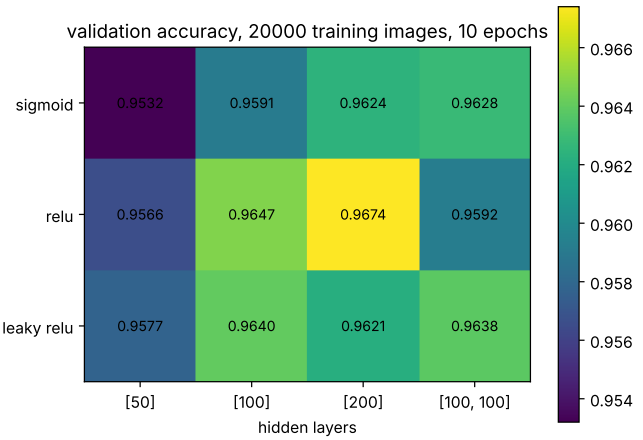


FIG. 9. **Activation function and hidden layers for MNIST.** Validation accuracy for the sigmoid, ReLU and Leaky ReLU with one hidden layer of 50, 100 or 200 nodes and with two hidden layers of 100 nodes (20 000 training images, 10 epochs).

The baseline for classification is logistic regression, which in our code is simply a network without hidden layers, $[784, 10]$, with a softmax output and cross-entropy cost. It is a convex model and linear in the pixels, so a network with hidden layers is only worthwhile if it clearly improves its performance. Its best learning rate with Adam is $\gamma = 3 \times 10^{-3}$, with a validation accuracy of 0.917.

For the network, Fig. 8 shows the validation accuracy of the $[784, 100, 10]$ network with ReLU as a function of γ and λ. The learning rate is more important than the penalty: $\gamma = 3 \times 10^{-4}$ is too slow for 10 epochs, while

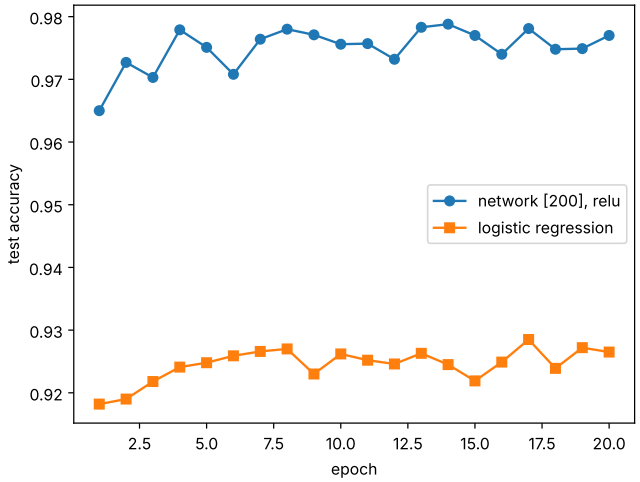


FIG. 10. **Training of the final models.** Test accuracy as a function of the epoch for the network $[784, 200, 10]$ and for logistic regression, both trained on the 60 000 training images. The curves only illustrate the training; no choice was made with the test set.

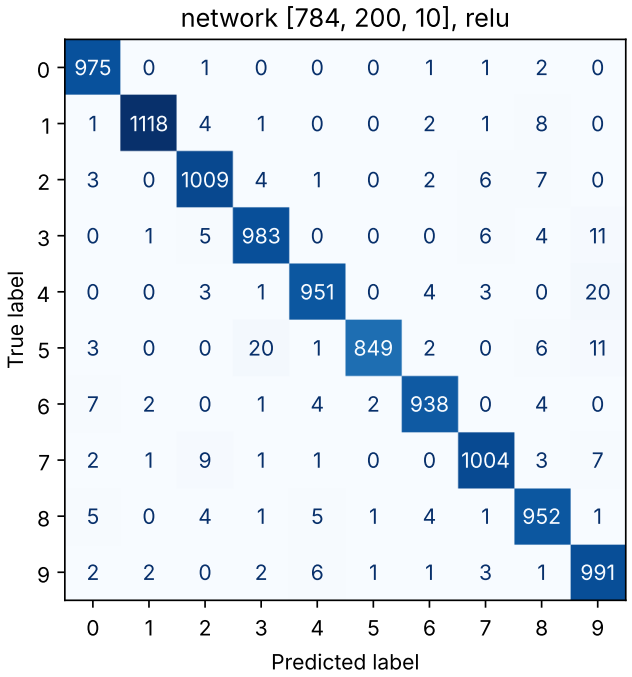


FIG. 11. **Confusion matrix** of the network $[784, 200, 10]$ with ReLU on the MNIST test set. Rows: true digit; columns: predicted digit. The diagonal contains the correctly classified images.

$\gamma = 10^{-2}$ is too large. A small L_2 penalty ($\lambda \leq 10^{-4}$) does not change the result, while $\lambda = 10^{-3}$ makes it worse. With 10 000 validation images, the standard error of an accuracy of 0.96 is $\sqrt{0.96 \cdot 0.04/10^4} \approx 0.002$, so the best cells (0.9639 and 0.9647) cannot be distinguished;

we choose $\gamma = 3 \times 10^{-3}$ and $\lambda = 10^{-5}$. Fig. 9 compares the three activation functions and several architectures. More neurons help slightly, from 50 to 200, and a second hidden layer does not improve the result with 20 000 images and 10 epochs. The best combination is ReLU with one layer of 200 neurons (0.9674), although the differences between activation functions are of the order of the standard error. We choose it because it performs best, because ReLU is computationally inexpensive, and because it does not suffer from vanishing gradients.

The final models are trained using all 60 000 training images for 20 epochs and evaluated only once on the test set (Table VI). The $[784, 200, 10]$ network with ReLU achieves an accuracy of 0.9770 ± 0.0015, compared with 0.9265 ± 0.0026 for logistic regression: a difference of about 5 percentage points, many times larger than the standard errors. The hidden layer learns combinations of pixels, such as strokes and shapes, that a model linear in the pixels cannot represent. Scikit-learn's `MLPClassifier`, with the same architecture and hyperparameters, gives 0.9784 ± 0.0015, in agreement with our network within the error, which confirms our implementation. A convolutional neural network would exceed 99 % [19], because it takes advantage of the two-dimensional structure of the images, which a dense network ignores by flattening them. Fig. 10 shows that the network reaches its final accuracy after a few epochs, whereas logistic regression levels off at around 92–93 %; this figure only illustrates the training process and was not used to make any selections.

The confusion matrix in Fig. 11 shows that all digits have precision and recall above 95 %. The most frequent errors are 4 classified as 9 and 5 classified as 3, with 20 cases each, followed by $5 \to 9$ and $3 \to 9$. These pairs have similar shapes: a 4 with a closed upper part resembles a 9, and the lower half of a 5 resembles that of a 3. The 5 has the lowest recall (0.952), meaning that it is the digit that is most often not recognized, and the 9 has the lowest precision (0.952), because many images of other digits are classified as 9. The 0 and 1 are almost never confused.

IV. CRITICAL EVALUATION OF THE METHODS

For regression, the polynomials from Project 1 are inexpensive, since they can be fitted in milliseconds using a closed-form solution or convex optimization, and they give low errors as long as they interpolate. Their weakness is at the edges of the interval, where a degree-12 polynomial blows up when extrapolating (Fig. 12, left). The networks achieve a CV MSE of about 0.012, close to the noise level, and are stable across all folds. In return, they have many more parameters, take seconds rather than milliseconds to train, have many hyperparameters to select, and their cost is non-convex, so the result depends on the initialization and optimizer. For a smooth one-dimensional function, the polynomial remains the best option if it is only used within the data range, because it is simpler and faster, whereas the network is safer near the edges.

As for the optimizers, standard gradient descent only works within a narrow range of learning rates and converges very slowly, whereas RMSprop and Adam with mini-batches work across an entire decade of γ, with Adam being the most robust. The learning rate is the most important hyperparameter. The architecture and activation function change the results by only one or two standard errors, and the penalties have little effect, whereas early stopping provides stronger regularization.

For classification, on the other hand, the network is clearly better than logistic regression (Fig. 12, right). Logistic regression has the advantage of being convex, fast, and easy to interpret, but it is linear in the pixels. The network learns its own features, which is exactly where its additional cost is justified [3]. Therefore, the best method for our regression problem is a well-chosen polynomial, while the best method for classification is the neural network. This agrees with the general perspective in Sec. 8.15 of Ref. [1]: neural networks are worthwhile when the data features are not known in advance.

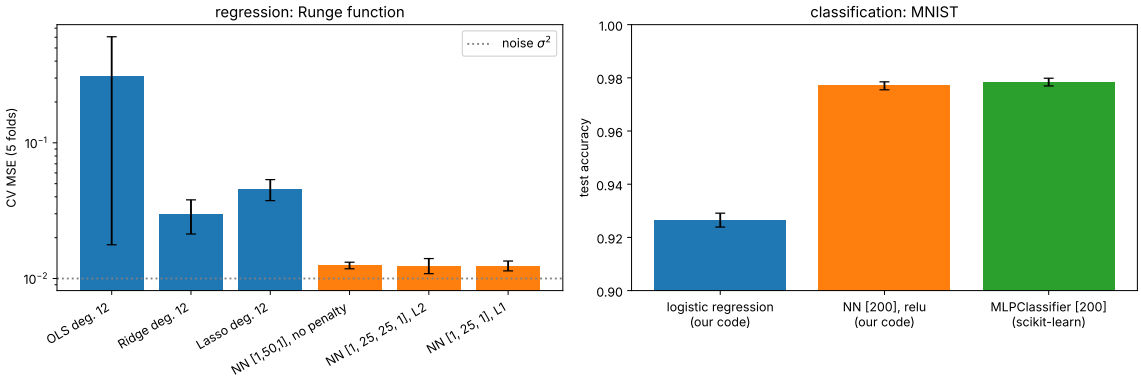


FIG. 12. **Summary of the results.** Left: cross-validated MSE of all the regression models on the same folds (blue: polynomials of project 1; orange: networks). Right: test accuracy on MNIST of logistic regression, our network and scikit-learn's `MLPClassifier`. The error bars are the standard errors.

V. CONCLUSION

We implemented our own dense neural network using backpropagation and applied it to a regression problem and a classification problem. For the regression of the Runge function, the network achieves a CV MSE of about 0.012, close to the noise level of 0.01. It is comparable to degree-12 OLS and Ridge, but much more stable near the edges, where the polynomial blows up when extrapolating. Adam is the most robust optimizer and the learning rate is the most important hyperparameter, whereas the architecture, activation function, and L_1 and L_2 penalties have only a small effect. The sigmoid performs slightly better for this smooth function, although its gradient vanishes in deep networks. For MNIST classification, a network with one hidden layer of 200 ReLU neurons reaches 97.7 %, five percentage points above logistic regression and in agreement with scikit-learn.

Our study has several limitations. The regression test set contains only 20 points, so its MSE has substantial uncertainty, and we used only one data split and one random seed. The grids of learning rates, penalties, and architectures are coarse, and the number of epochs was fixed. For MNIST, the searches were performed using 20 000 images and a single validation set instead of cross-validation, and we did not carry out a bias-variance analysis of the networks.

As future work, we would implement automatic early stopping using a validation set, rather than fixing the number of epochs, and use He initialization for ReLU networks. Repeating the experiments with several random seeds would make it possible to provide error bars for the test results as well. For MNIST, a convolutional neural network would take advantage of the image structure and should exceed 99 % accuracy; it would also be interesting to apply the same code to a more challenging dataset, such as Fashion-MNIST.

Appendix A: Use of AI/LLM tools

A.1 Tools used. Claude (Anthropic), used through the Claude application, in November 2026.

A.2 Text writing and editing.

Section	Level	Notes
Abstract	1	Claude has helped me translate it into English and correct any spelling mistakes.
Introduction	1	Claude has helped me translate it into English and correct any spelling mistakes.
Methods (theory of part a)	2	The equations and everything else are my own, but Claude has helped me convert everything to LaTeX so that it all looks better.
Methods (rest)	2	The equations and everything else are my own, but Claude has helped me convert everything to LaTeX so that it all looks better.
Results and discussion	1	Claude has helped me translate it into English and correct any spelling mistakes.

Critical evaluation	1	Claude has helped me translate it into English and correct any spelling mistakes.
Conclusion	1	Claude has helped me translate it into English and correct any spelling mistakes.
Report skeleton (LaTeX)	3	I asked Claude to draw up the basic outline of the report for me: the scientific format, the two columns, how to cite sources, the sections it should include... and I then filled it in with my own data.

A.3 Code generation and assistance.

Part / cell	Level	Description
a) functions	2	Claude made the functions from my notes
b)	3	Claude has helped me with some parts of the code that I couldn't quite get right, but mainly he's helped to correct errors and make the graphs look better.
c)	2	Claude helped me spot quite a few mistakes, explained them to me and made sure the graphs looked right
d)	1	Claude has helped me correct mistakes and offer a few comments.
e)	2	Claude helped me create some functions and loops, as well as a few short blocks. I tested them and integrated them.
f)	1	Claude helped me fix a few bugs and make sure the printouts and graphs look good

* albertg@uio.no

[1] M. Hjorth-Jensen, *Applied Data Analysis and Machine Learning*, lecture notes, University of Oslo (2026), https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/intro.html.

[2] A. García Asensio, *Regression analysis of the Runge function: OLS, Ridge and Lasso, resampling methods and gradient descent*, Project 1, FYS-STK3155, University of Oslo (2026).

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, Cambridge, MA, 2016), https://www.deeplearningbook.org/.

[4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York, 2009).

[5] M. A. Nielsen, *Neural Networks and Deep Learning* (Determination Press, 2015), http://neuralnetworksanddeeplearning.com/.

[6] C. Runge, *Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten*, Z. Math. Phys. **46**, 224 (1901).

[7] A. E. Hoerl and R. W. Kennard, *Ridge regression: Biased estimation for nonorthogonal problems*, Technometrics **12**, 55 (1970).

[8] R. Tibshirani, *Regression shrinkage and selection via the lasso*, J. R. Stat. Soc. B **58**, 267 (1996).

[9] K. Hornik, M. Stinchcombe, and H. White, *Multilayer feedforward networks are universal approximators*, Neural Netw. **2**, 359 (1989).

[10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, *Learning representations by back-propagating errors*, Nature **323**, 533 (1986).

[11] X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, in Proc. 13th Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR **9**, 249 (2010).

[12] K. He, X. Zhang, S. Ren, and J. Sun, *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*, in Proc. IEEE Int. Conf. Computer Vision (ICCV), 1026 (2015).

[13] V. Nair and G. E. Hinton, *Rectified linear units improve restricted Boltzmann machines*, in Proc. 27th Int. Conf. Machine Learning (ICML), 807 (2010).

[14] A. L. Maas, A. Y. Hannun, and A. Y. Ng, *Rectifier nonlinearities improve neural network acoustic models*, in ICML Workshop on Deep Learning for Audio, Speech and Language Processing (2013).

[15] Y. Bengio, P. Simard, and P. Frasconi, *Learning long-term dependencies with gradient descent is difficult*, IEEE Trans. Neural Netw. **5**, 157 (1994).

[16] L. Prechelt, *Early stopping, but when?*, in *Neural Networks: Tricks of the Trade*, edited by G. B. Orr and K.-R. Müller (Springer, Berlin, 1998), p. 55.

[17] T. Tieleman and G. Hinton, *Lecture 6.5, rmsprop: Divide*

the gradient by a running average of its recent magnitude, COURSERA: Neural Networks for Machine Learning (2012).

[18] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, in Proc. 3rd Int. Conf. Learning Representations (ICLR) (2015), arXiv:1412.6980.

[19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based learning applied to document recognition*, Proc. IEEE **86**, 2278 (1998).

[20] Y. LeCun, C. Cortes, and C. J. C. Burges, *The MNIST database of handwritten digits*, http://yann.lecun.com/exdb/mnist/.

[21] F. Pedregosa *et al.*, *Scikit-learn: Machine learning in Python*, J. Mach. Learn. Res. **12**, 2825 (2011).

[22] C. R. Harris *et al.*, *Array programming with NumPy*, Nature **585**, 357 (2020).

[23] J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).